ASSIGNMENT 22

Name: Om Prashant Kulawade

College: Zeal Polytechnic, Narhe, Pune

Branch: Artificial Intelligence and Machine Learning (AIML)

Domain: AIML (Artificial Intelligence and Machine Learning)

https://github.com/omkulawade03/Frontend-Session-Assignments.git

---

Dataset Used

Dataset Name:
Breast Cancer Wisconsin Diagnostic Dataset

Dataset Link:
https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data

---

Import Required Libraries

```
# ============================================================
# Import Required Libraries
# ============================================================

# Import Pandas for data loading and manipulation
import pandas as pd

# Import NumPy for numerical operations
import numpy as np

# Import Matplotlib for plotting graphs
import matplotlib.pyplot as plt

# Import Seaborn for statistical data visualization
import seaborn as sns

# Import Joblib to save and load trained ML models
import joblib

# Import train_test_split to split dataset into training and testing sets
from sklearn.model_selection import train_test_split

# Import StandardScaler for feature scaling
from sklearn.preprocessing import StandardScaler

# Import Logistic Regression algorithm
from sklearn.linear_model import LogisticRegression

# Import evaluation metrics
from sklearn.metrics import (
    confusion_matrix,      # Confusion Matrix
    accuracy_score,        # Accuracy
    precision_score,       # Precision
    recall_score,          # Recall
    f1_score,              # F1 Score
    classification_report  # Classification Report
)

# Display success message
print("All required libraries imported successfully.")
```

[1] ✓ 5.3s

```
All required libraries imported successfully.
```

Q1. (Dataset Loading)

- **Load the Breast Cancer dataset (data.csv).**
- **Display the first 5 records.**
- **Display dataset information using info().**
- **Display the dataset shape.**

Program Code:

#Questions 01

```
# ============================================================
# Q1. Dataset Loading
# ============================================================

# Load the Breast Cancer dataset
df = pd.read_csv("data.csv")

# Display the first 5 records
print("========== First 5 Records ==========")
print(df.head())

# Display dataset information
print("\n========== Dataset Information ==========")
df.info()

# Display the shape of the dataset
print("\n========== Dataset Shape ==========")
print(df.shape)
```

[5] ✓ 0.0s

OUTPUT SCREENSHOT:

```
========== First 5 Records ==========
         id diagnosis  radius_mean  texture_mean  perimeter_mean  area_mean  \
0    842302         M        17.99         10.38          122.80     1001.0
1    842517         M        20.57         17.77          132.90     1326.0
2  84300903         M        19.69         21.25          130.00     1203.0
3  84348301         M        11.42         20.38           77.58      386.1
4  84358402         M        20.29         14.34          135.10     1297.0

   smoothness_mean  compactness_mean  concavity_mean  concave points_mean  \
0          0.11840           0.27760          0.3001              0.14710
1          0.08474           0.07864          0.0869              0.07017
2          0.10960           0.15990          0.1974              0.12790
3          0.14250           0.28390          0.2414              0.10520
4          0.10030           0.13280          0.1980              0.10430

   ...  texture_worst  perimeter_worst  area_worst  smoothness_worst  \
0  ...          17.33           184.60      2019.0            0.1622
1  ...          23.41           158.80      1956.0            0.1238
2  ...          25.53           152.50      1709.0            0.1444
3  ...          26.50            98.87       567.7            0.2098
4  ...          16.67           152.20      1575.0            0.1374

   compactness_worst  concavity_worst  concave points_worst  symmetry_worst  \
0             0.6656           0.7119                0.2654          0.4601
1             0.1866           0.2416                0.1860          0.2750
...
memory usage: 146.8 KB
```

```
========== Dataset Information ==========
<class 'pandas.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 33 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   id                      569 non-null    int64
 1   diagnosis               569 non-null    str
 2   radius_mean             569 non-null    float64
 3   texture_mean            569 non-null    float64
 4   perimeter_mean          569 non-null    float64
 5   area_mean               569 non-null    float64
 6   smoothness_mean         569 non-null    float64
 7   compactness_mean        569 non-null    float64
 8   concavity_mean          569 non-null    float64
 9   concave points_mean     569 non-null    float64
 10  symmetry_mean           569 non-null    float64
 11  fractal_dimension_mean  569 non-null    float64
 12  radius_se               569 non-null    float64
 13  texture_se              569 non-null    float64
 14  perimeter_se            569 non-null    float64
 15  area_se                 569 non-null    float64
 16  smoothness_se           569 non-null    float64
 17  compactness_se          569 non-null    float64
...
memory usage: 146.8 KB
```

```
========== Dataset Shape ==========
(569, 33)
```

Q2. (Data Cleaning)

- **Check for missing values.**
- **Remove unnecessary columns (if any).**
- **Handle duplicate records.**
- **Verify the cleaned dataset.**

Program Code:

#Questions 02

```
# ============================================================
# Q2. Data Cleaning
# ============================================================

# Check for missing values in each column
print("========== Missing Values ==========")
print(df.isnull().sum())

# Remove unnecessary columns (if present)
# In this dataset, the 'id' column is not useful for prediction
if "id" in df.columns:
    df.drop("id", axis=1, inplace=True)
    print("\n'id' column removed successfully.")

# Check for duplicate records
print("\n========== Duplicate Records ==========")
print("Number of duplicate rows:", df.duplicated().sum())

# Remove duplicate records
df.drop_duplicates(inplace=True)

print("Duplicate records removed successfully.")

# Verify the cleaned dataset
print("\n========== Cleaned Dataset ==========")
print(df.head())

print("\n========== Dataset Shape After Cleaning ==========")
print(df.shape)
```

[7] ✓ 0.0s

OUTPUT SCREENSHOT:

```
========== Missing Values ==========
id                          0
diagnosis                   0
radius_mean                 0
texture_mean                0
perimeter_mean              0
area_mean                   0
smoothness_mean             0
compactness_mean            0
concavity_mean              0
concave points_mean         0
symmetry_mean               0
fractal_dimension_mean      0
radius_se                   0
texture_se                  0
perimeter_se                0
area_se                     0
smoothness_se               0
compactness_se              0
concavity_se                0
concave points_se           0
symmetry_se                 0
fractal_dimension_se        0
radius_worst                0
texture_worst               0
...
[5 rows x 32 columns]
```

```
========== Duplicate Records ==========
Number of duplicate rows: 0
Duplicate records removed successfully.

========== Cleaned Dataset ==========
  diagnosis  radius_mean  texture_mean  perimeter_mean  area_mean  \
0         M        17.99         10.38          122.80     1001.0
1         M        20.57         17.77          132.90     1326.0
2         M        19.69         21.25          130.00     1203.0
3         M        11.42         20.38           77.58      386.1
4         M        20.29         14.34          135.10     1297.0

   smoothness_mean  compactness_mean  concavity_mean  concave points_mean  \
0          0.11840           0.27760          0.3001              0.14710
1          0.08474           0.07864          0.0869              0.07017
2          0.10960           0.15990          0.1974              0.12790
3          0.14250           0.28390          0.2414              0.10520
4          0.10030           0.13280          0.1980              0.10430

   symmetry_mean  ...  texture_worst  perimeter_worst  area_worst  \
0         0.2419  ...          17.33           184.60      2019.0
1         0.1812  ...          23.41           158.80      1956.0
2         0.2069  ...          25.53           152.50      1709.0
3         0.2597  ...          26.50            98.87       567.7
...
[5 rows x 32 columns]

========== Dataset Shape After Cleaning ==========
(569, 32)
```

Q3. (Exploratory Data Analysis - EDA)

- Display summary statistics.
- Plot class distribution.
- Plot correlation heatmap.
- Visualize important features using histograms or boxplots.

Program Code:

```
#Questions 03
```

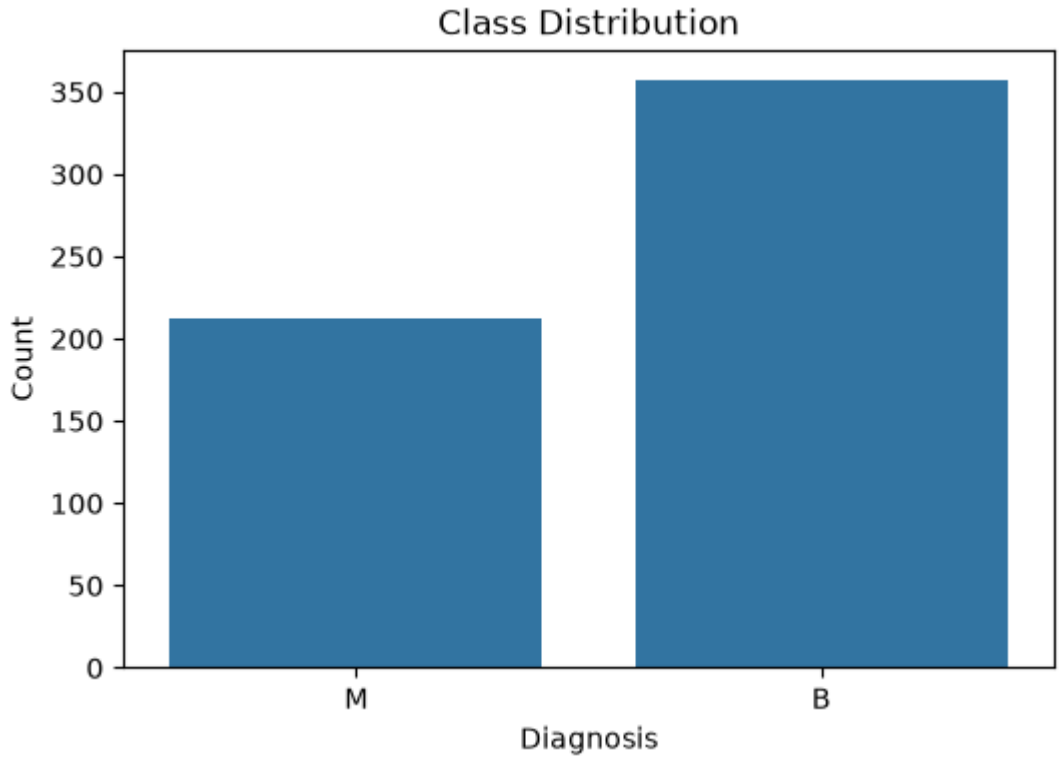
```python
# ============================================================
# Q3. Exploratory Data Analysis (EDA)
# ============================================================

# Display summary statistics
print("========== Summary Statistics ==========")
print(df.describe())

# ------------------------------------------------------------
# Plot Class Distribution
# ------------------------------------------------------------

plt.figure(figsize=(6,4))
sns.countplot(x='diagnosis', data=df)
plt.title("Class Distribution")
plt.xlabel("Diagnosis")
plt.ylabel("Count")
plt.show()

# ------------------------------------------------------------
# Plot Correlation Heatmap
# ------------------------------------------------------------

plt.figure(figsize=(14,10))
sns.heatmap(df.drop('diagnosis', axis=1).corr(),
            cmap='coolwarm',
            linewidths=0.5)
plt.title("Correlation Heatmap")
plt.show()
```

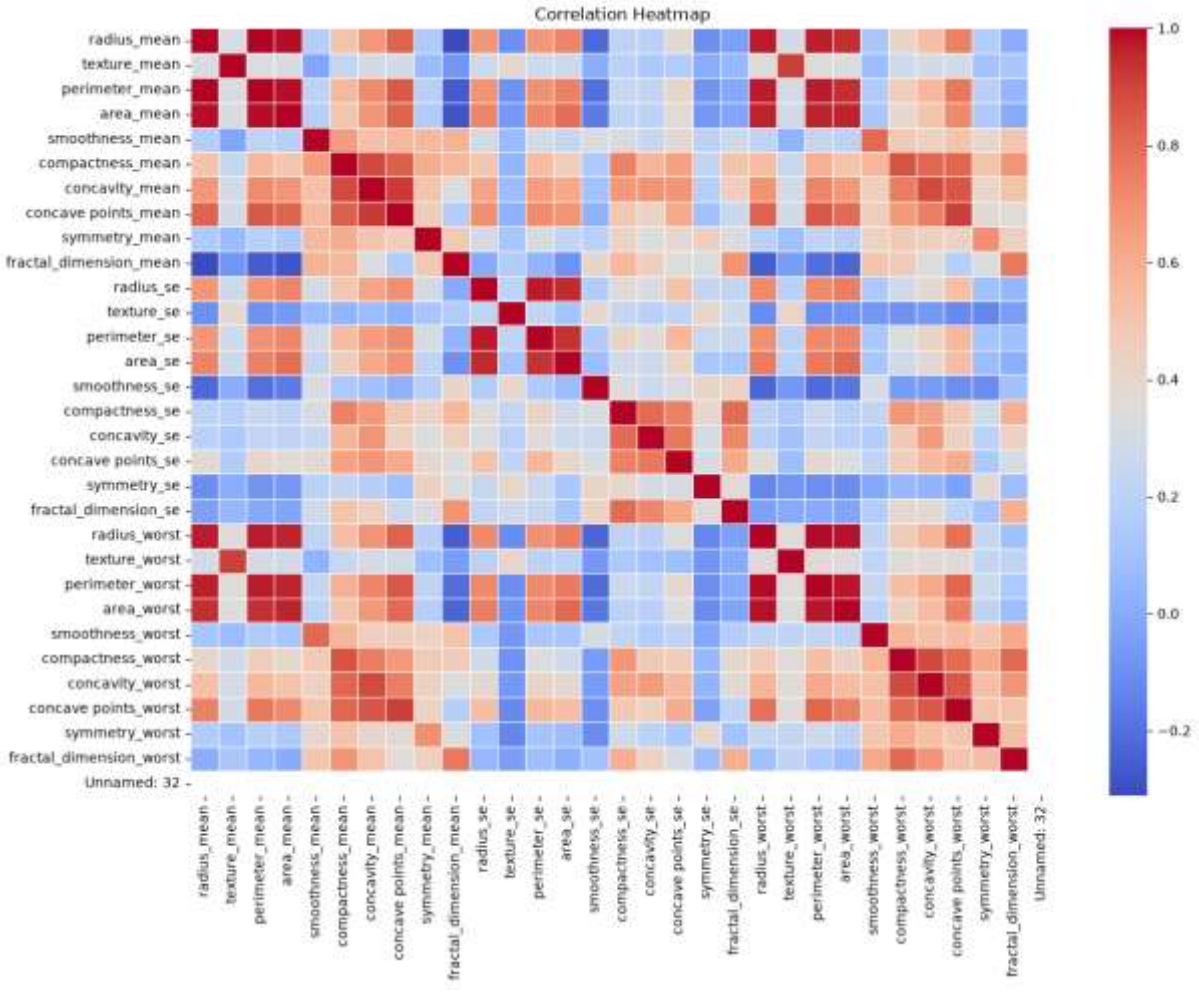
```
# ------------------------------------------------------------
# Histogram of Important Feature
# ------------------------------------------------------------

plt.figure(figsize=(6,4))
sns.histplot(df['radius_mean'], bins=30, kde=True)
plt.title("Distribution of Radius Mean")
plt.xlabel("Radius Mean")
plt.ylabel("Frequency")
plt.show()

# ------------------------------------------------------------
# Boxplot of Important Feature
# ------------------------------------------------------------

plt.figure(figsize=(6,4))
sns.boxplot(x='diagnosis', y='radius_mean', data=df)
plt.title("Radius Mean vs Diagnosis")
plt.xlabel("Diagnosis")
plt.ylabel("Radius Mean")
plt.show()
```

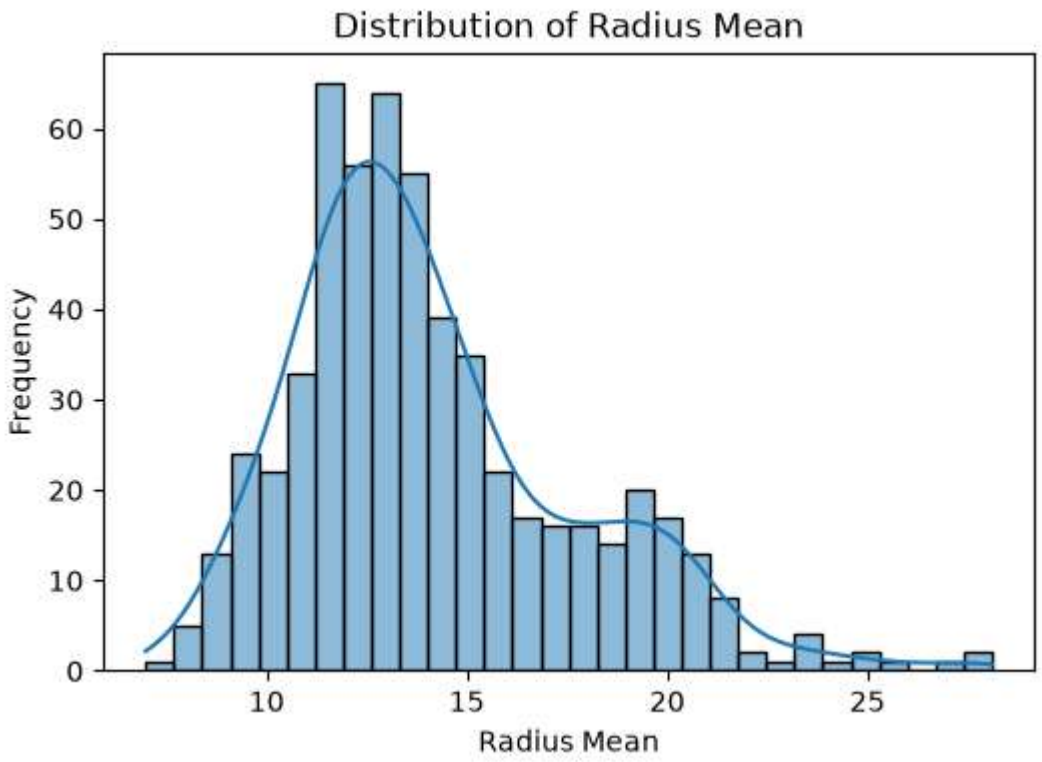
[12] ✓ 1.0s

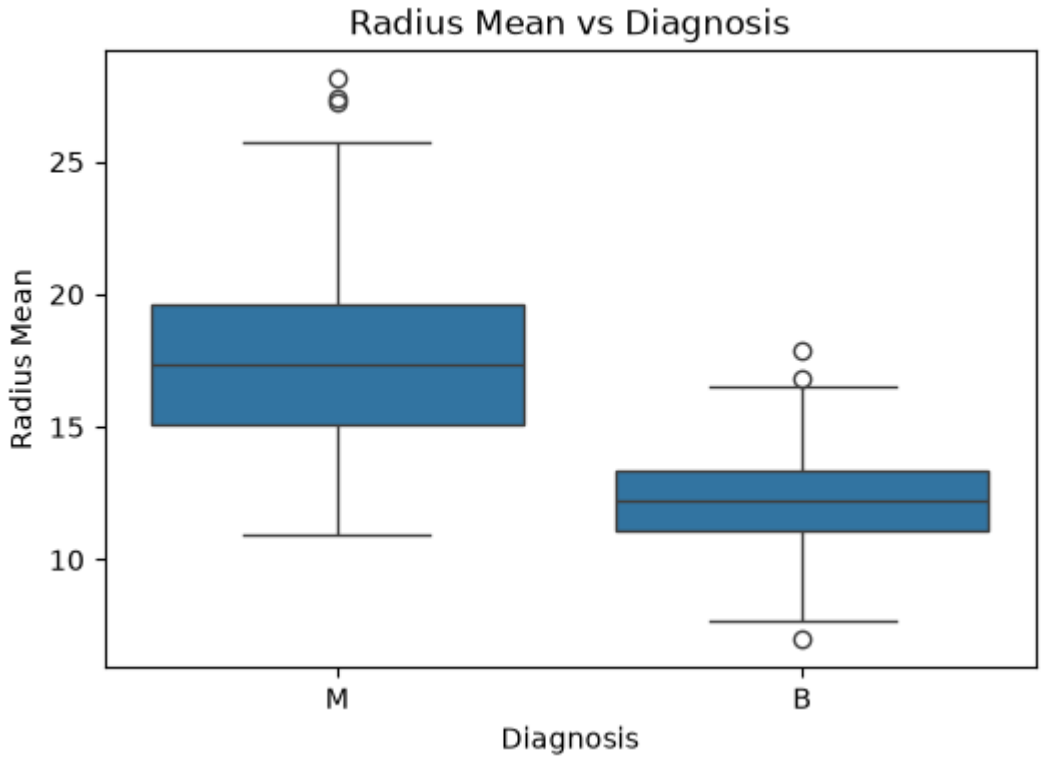
OUTPUT SCREENSHOT:

```
...   ========== Summary Statistics ==========
           radius_mean  texture_mean  perimeter_mean    area_mean  \
      count   569.000000    569.000000      569.000000   569.000000
      mean     14.127292     19.289649       91.969033   654.889104
      std       3.524049      4.301036       24.298981   351.914129
      min       6.981000      9.710000       43.790000   143.500000
      25%      11.700000     16.170000       75.170000   420.300000
      50%      13.370000     18.840000       86.240000   551.100000
      75%      15.780000     21.800000      104.100000   782.700000
      max      28.110000     39.280000      188.500000  2501.000000

           smoothness_mean  compactness_mean  concavity_mean  concave points_mean  \
      count       569.000000        569.000000      569.000000           569.000000
      mean          0.096360          0.104341        0.088799             0.048919
      std           0.014064          0.052813        0.079720             0.038803
      min           0.052630          0.019380        0.000000             0.000000
      25%           0.086370          0.064920        0.029560             0.020310
      50%           0.095870          0.092630        0.061540             0.033500
      75%           0.105300          0.130400        0.130700             0.074000
      max           0.163400          0.345400        0.426800             0.201200

           symmetry_mean  fractal_dimension_mean  ...  texture_worst  \
      count     569.000000              569.000000  ...     569.000000
      mean        0.181162                0.062798  ...      25.677223
      std         0.027414                0.007060  ...       6.146258
      ...
      75%                 0.092080          NaN
      max                 0.207500          NaN

      [8 rows x 31 columns]
```


Class Distribution
Count
350
300
250
200
150
100
50
0
M
B
Diagnosis


Correlation Heatmap
radius_mean
texture_mean
perimeter_mean
area_mean
smoothness_mean
compactness_mean
concavity_mean
concave points_mean
symmetry_mean
fractal_dimension_mean
radius_se
texture_se
perimeter_se
area_se
smoothness_se
compactness_se
concavity_se
concave points_se
symmetry_se
fractal_dimension_se
radius_worst
texture_worst
perimeter_worst
area_worst
smoothness_worst
compactness_worst
concavity_worst
concave points_worst
symmetry_worst
fractal_dimension_worst
Unnamed: 32
1.0
0.8
0.6
0.4
0.2
0.0
−0.2


Distribution of Radius Mean
Frequency
0
10
20
30
40
50
60
10
15
20
25
Radius Mean


Radius Mean vs Diagnosis
Radius Mean
10
15
20
25
M
B
Diagnosis

Q4. (Feature Selection & Preprocessing)

- **Separate independent variables (X) and target variable (y).**
- **Encode the target variable if required.**
- **Scale numerical features using StandardScaler.**

Program Code:

```
#Questions 04
```

```python
# ============================================================
# Q4. Feature Selection & Preprocessing
# ============================================================

# Separate independent variables (X) and target variable (y)
print("========== Separating Features and Target ==========")

X = df.drop("diagnosis", axis=1)
y = df["diagnosis"]

print("\nIndependent Variables (X):")
print(X.head())

print("\nTarget Variable (y):")
print(y.head())

# ------------------------------------------------------------

# Encode the target variable
print("\n========== Encoding Target Variable ==========")

from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()

y = encoder.fit_transform(y)

print("Encoded Target Values:")
print(y[:10])
```

```python
# ------------------------------------------------------------

# Scale numerical features
print("\n========== Scaling Numerical Features ==========")

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X = scaler.fit_transform(X)

print("Scaled Feature Values:")
print(X[:5])
```

[14] ✓ 0.0s

OUTPUT SCREENSHOT:

```
========== Separating Features and Target ==========

Independent Variables (X):
   radius_mean  texture_mean  perimeter_mean  area_mean  smoothness_mean  \
0        17.99         10.38          122.80     1001.0          0.11840
1        20.57         17.77          132.90     1326.0          0.08474
2        19.69         21.25          130.00     1203.0          0.10960
3        11.42         20.38           77.58      386.1          0.14250
4        20.29         14.34          135.10     1297.0          0.10030

   compactness_mean  concavity_mean  concave points_mean  symmetry_mean  \
0           0.27760          0.3001              0.14710         0.2419
1           0.07864          0.0869              0.07017         0.1812
2           0.15990          0.1974              0.12790         0.2069
3           0.28390          0.2414              0.10520         0.2597
4           0.13280          0.1980              0.10430         0.1809

   fractal_dimension_mean  ...  texture_worst  perimeter_worst  area_worst  \
0                 0.07871  ...          17.33           184.60      2019.0
1                 0.05667  ...          23.41           158.80      1956.0
2                 0.05999  ...          25.53           152.50      1709.0
3                 0.09744  ...          26.50            98.87       567.7
4                 0.05883  ...          16.67           152.20      1575.0

   smoothness_worst  compactness_worst  concavity_worst  concave points_worst  \
...
2    M
3    M
4    M
Name: diagnosis, dtype: str
```

```
...
========== Encoding Target Variable ==========
Encoded Target Values:
[1 1 1 1 1 1 1 1 1 1]
```

```
========== Scaling Numerical Features ==========
Scaled Feature Values:
[[ 1.09706398e+00 -2.07333501e+00  1.26993369e+00  9.84374905e-01
   1.56846633e+00  3.28351467e+00  2.65287398e+00  2.53247522e+00
   2.21751501e+00  2.25574689e+00  2.48973393e+00 -5.65265059e-01
   2.83303087e+00  2.48757756e+00 -2.14001647e-01  1.31686157e+00
   7.24026158e-01  6.60819941e-01  1.14875667e+00  9.07083081e-01
   1.88668963e+00 -1.35929347e+00  2.30360062e+00  2.00123749e+00
   1.30768627e+00  2.61666502e+00  2.10952635e+00  2.29607613e+00
   2.75062224e+00  1.93701461e+00             nan]
 [ 1.82982061e+00 -3.53632408e-01  1.68595471e+00  1.90870825e+00
  -8.26962447e-01 -4.87071673e-01 -2.38458552e-02  5.48144156e-01
   1.39236330e-03 -8.68652457e-01  4.99254601e-01 -8.76243603e-01
   2.63326966e-01  7.42401948e-01 -6.05350847e-01 -6.92926270e-01
  -4.40780058e-01  2.60162067e-01 -8.05450380e-01 -9.94437403e-02
   1.80592744e+00 -3.69203222e-01  1.53512599e+00  1.89048899e+00
  -3.75611957e-01 -4.30444219e-01 -1.46748968e-01  1.08708430e+00
  -2.43889668e-01  2.81189987e-01             nan]
 [ 1.57988811e+00  4.56186952e-01  1.56650313e+00  1.55888363e+00
   9.42210440e-01  1.05292554e+00  1.36347845e+00  2.03723076e+00
   9.39684817e-01 -3.98007910e-01  1.22867595e+00 -7.80083377e-01
   8.50928301e-01  1.18133606e+00 -2.97005012e-01  8.14973504e-01
   2.13076435e-01  1.42482747e+00  2.37035535e-01  2.93559404e-01
   1.51187025e+00 -2.39743838e-02  1.34747521e+00  1.45628455e+00
...
   8.28470780e-01  1.14420474e+00 -3.61092272e-01  4.99328134e-01
   1.29857524e+00 -1.46677038e+00  1.33853946e+00  1.22072425e+00
   2.20556166e-01 -3.13394511e-01  6.13178758e-01  7.29259257e-01
  -8.68352984e-01 -3.97099619e-01             nan]]
```

Q5. (Train-Test Split)

- **Split the dataset into training and testing sets.**
- **Use:**
  - **test_size = 0.20**
  - **random_state = 42**
- **Print the shapes of:**
  - **X_train**
  - **X_test**
  - **y_train**
  - **y_test**

Program Code:

```
#Questions 05
```

```
# ============================================================
# Q5. Train-Test Split
# ============================================================

# Split the dataset into training and testing sets
print("========== Train-Test Split ==========")

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

# Display the shapes of training and testing data
print("\nShape of X_train:")
print(X_train.shape)

print("\nShape of X_test:")
print(X_test.shape)

print("\nShape of y_train:")
print(y_train.shape)

print("\nShape of y_test:")
print(y_test.shape)
```

[19] ✓ 0.0s

OUTPUT SCREENSHOT:

```
========== Train-Test Split ==========

Shape of X_train:
(455, 31)

Shape of X_test:
(114, 31)

Shape of y_train:
(455,)

Shape of y_test:
(114,)
```

Q6. (Model Building)

- **Import LogisticRegression.**
- **Create the Logistic Regression model.**
- **Train the model using the training dataset.**

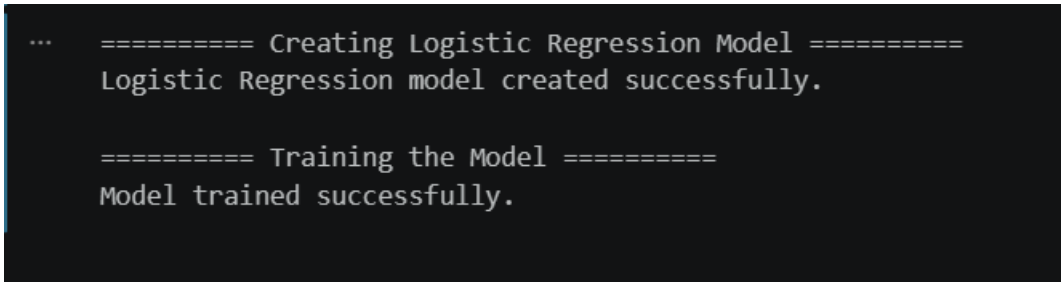
Program Code:

#Questions 06

```
# ============================================================
# Q6. Model Building
# ============================================================

# Import Logistic Regression
from sklearn.linear_model import LogisticRegression

# Create the Logistic Regression model
print("========== Creating Logistic Regression Model ==========")

model = LogisticRegression(max_iter=1000)

print("Logistic Regression model created successfully.")

# Train the model using training data
print("\n========== Training the Model ==========")

model.fit(X_train, y_train)

print("Model trained successfully.")
```

[35] ✓ 0.0s

OUTPUT SCREENSHOT:



Q7. (Prediction)

- **Predict the test dataset.**
- **Store predictions in y_pred.**
- **Display the first 10 actual and predicted values.**

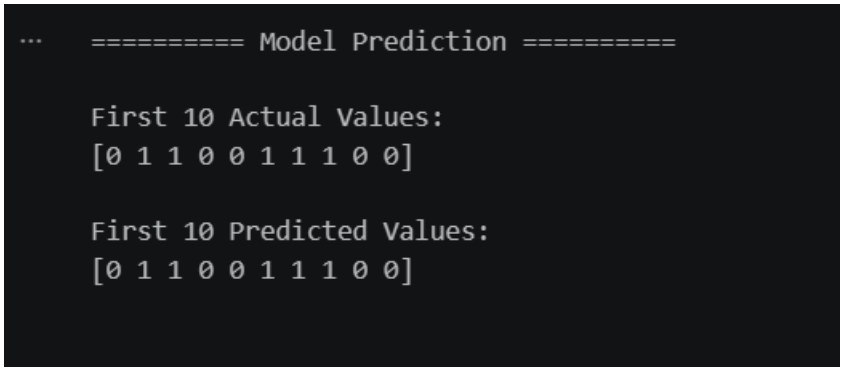
Program Code:

#Questions 07

```
# ============================================================
# Q7. Prediction
# ============================================================

# Predict the test dataset
print("========== Model Prediction ==========")

# Store predictions in y_pred
y_pred = model.predict(X_test)

# Display first 10 actual values
print("\nFirst 10 Actual Values:")
print(y_test[:10])

# Display first 10 predicted values
print("\nFirst 10 Predicted Values:")
print(y_pred[:10])
```

[37] ✓ 0.0s

OUTPUT SCREENSHOT:



Q8. (Model Evaluation)

Calculate and display:

- **Accuracy**
- **Precision**
- **Recall**
- **F1 Score**

Also display:

- **Confusion Matrix**
- **Classification Report**

Program Code:

```
#Questions 08
```

```python
# ============================================================
# Q8. Model Evaluation
# ============================================================

# Import Evaluation Metrics
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

print("========== Model Evaluation ==========")

# ----------------------------------------
# Accuracy
# ----------------------------------------
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:")
print(accuracy)

# ----------------------------------------
# Precision
# ----------------------------------------
precision = precision_score(y_test, y_pred)
print("\nPrecision:")
print(precision)
```

```python
# ----------------------------------------
# Recall
# ----------------------------------------
recall = recall_score(y_test, y_pred)
print("\nRecall:")
print(recall)

# ----------------------------------------
# F1 Score
# ----------------------------------------
f1 = f1_score(y_test, y_pred)
print("\nF1 Score:")
print(f1)

# ----------------------------------------
# Confusion Matrix
# ----------------------------------------
cm = confusion_matrix(y_test, y_pred)

print("\n========== Confusion Matrix ==========")
print(cm)

# ----------------------------------------
# Classification Report
# ----------------------------------------
print("\n========== Classification Report ==========")
print(classification_report(y_test, y_pred))
```

[38] ✓ 0.0s

OUTPUT SCREENSHOT:

```
========== Model Evaluation ==========

Accuracy:
0.9736842105263158

Precision:
0.9761904761904762

Recall:
0.9534883720930233

F1 Score:
0.9647058823529412

========== Confusion Matrix ==========
[[70  1]
 [ 2 41]]

========== Classification Report ==========
              precision    recall  f1-score   support

           0       0.97      0.99      0.98        71
           1       0.98      0.95      0.96        43

    accuracy                           0.97       114
   macro avg       0.97      0.97      0.97       114
weighted avg       0.97      0.97      0.97       114
```

Q9. (Save the Model)

Save the following using joblib:

- **Model → breast_cancer_model.pkl**
- **Scaler → scaler.pkl**
- **Feature Columns → columns.pkl**

Program Code:

#Questions 09

```
# ============================================================
# Q9. Save the Model
# ============================================================

# Import Joblib
import joblib

print("========== Saving Model ==========")

# Save the trained Logistic Regression model
joblib.dump(model, "breast_cancer_model.pkl")

# Save the StandardScaler object
joblib.dump(scaler, "scaler.pkl")

# Save the feature (column) names
joblib.dump(df.drop("diagnosis", axis=1).columns.tolist(), "columns.pkl")

print("\nModel saved as: breast_cancer_model.pkl")
print("Scaler saved as: scaler.pkl")
print("Feature columns saved as: columns.pkl")

print("\nAll files saved successfully.")
```

[39] ✓ 0.0s

OUTPUT SCREENSHOT:

```
========== Saving Model ==========

Model saved as: breast_cancer_model.pkl
Scaler saved as: scaler.pkl
Feature columns saved as: columns.pkl

All files saved successfully.
```

Q10. (Load Saved Model)

- **Load the saved model.**
- **Create one sample patient record.**
- **Preprocess the input.**
- **Predict whether the tumor is:**
  - **Benign**
  - **Malignant**

Program Code:

#Questions 10

```
# ============================================================
# Q10. Load Saved Model
# ============================================================

# Import Required Libraries
import pandas as pd
import joblib

print("========== Loading Saved Model ==========")

# ----------------------------------------
# Load Saved Model and Objects
# ----------------------------------------

model = joblib.load("breast_cancer_model.pkl")
scaler = joblib.load("scaler.pkl")
columns = joblib.load("columns.pkl")

print("Model Loaded Successfully.")
```

```python
# ----------------------------------------
# Create One Sample Patient Record
# ----------------------------------------

sample = {
    "radius_mean": 17.99,
    "texture_mean": 10.38,
    "perimeter_mean": 122.8,
    "area_mean": 1001.0,
    "smoothness_mean": 0.1184,
    "compactness_mean": 0.2776,
    "concavity_mean": 0.3001,
    "concave points_mean": 0.1471,
    "symmetry_mean": 0.2419,
    "fractal_dimension_mean": 0.07871,
    "radius_se": 1.095,
    "texture_se": 0.9053,
    "perimeter_se": 8.589,
    "area_se": 153.4,
    "smoothness_se": 0.006399,
    "compactness_se": 0.04904,
    "concavity_se": 0.05373,
    "concave points_se": 0.01587,
    "symmetry_se": 0.03003,
    "fractal_dimension_se": 0.006193,
    "radius_worst": 25.38,
    "texture_worst": 17.33,
    "perimeter_worst": 184.6,
    "area_worst": 2019.0,
    "smoothness_worst": 0.1622,
    "compactness_worst": 0.6656,
    "concavity_worst": 0.7119,
    "concave points_worst": 0.2654,
    "symmetry_worst": 0.4601,
    "fractal_dimension_worst": 0.1189
}
```

```
# Convert dictionary to DataFrame
sample_df = pd.DataFrame([sample])

# Arrange columns in the same order as training
sample_df = sample_df[columns]

# ----------------------------------------
# Preprocess Input
# ----------------------------------------

sample_scaled = scaler.transform(sample_df)

# ----------------------------------------
# Predict the Tumor Type
# ----------------------------------------

prediction = model.predict(sample_scaled)

print("\n========== Prediction Result ==========")

if prediction[0] == 1:
    print("Tumor Prediction: Malignant")
else:
    print("Tumor Prediction: Benign")
```

[40] ✓ 0.0s

OUTPUT SCREENSHOT:

```
========== Loading Saved Model ==========
Model Loaded Successfully.

========== Prediction Result ==========
Tumor Prediction: Malignant
```

Q11. (Streamlit Input Interface)

Create a Streamlit application (app.py) with:

- **st.number_input() for numerical features.**
- **st.button("Predict").**

Program Code:

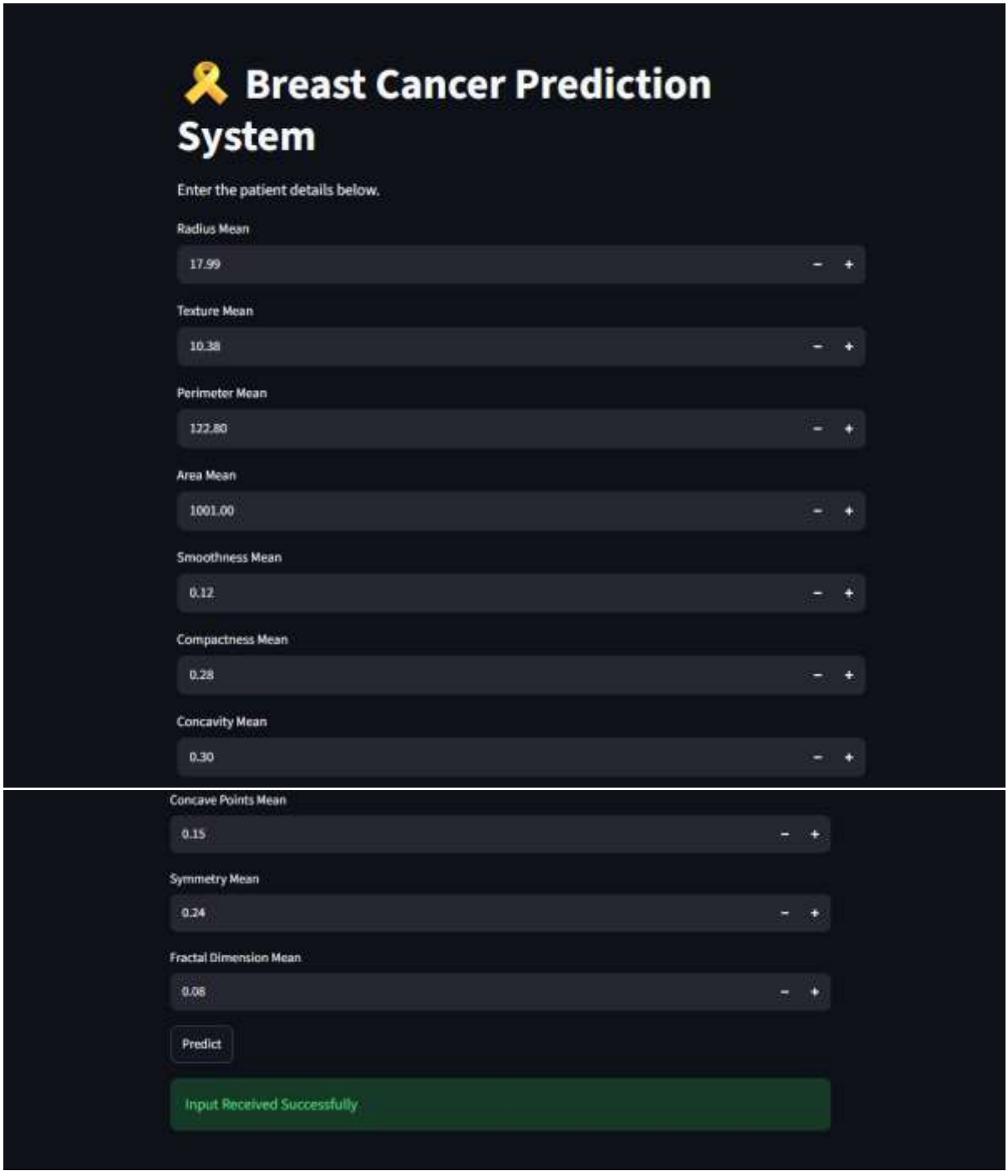
#Questions 11

app.py > ...

```python
# ============================================================
# Q11. Streamlit Input Interface
# ============================================================

import streamlit as st

# Page Title
st.set_page_config(page_title="Breast Cancer Prediction")

# Heading
st.title("🎗️ Breast Cancer Prediction System")

st.write("Enter the patient details below.")

# ----------------------------
# Input Fields
# ----------------------------

radius_mean = st.number_input("Radius Mean", value=17.99)

texture_mean = st.number_input("Texture Mean", value=10.38)

perimeter_mean = st.number_input("Perimeter Mean", value=122.8)

area_mean = st.number_input("Area Mean", value=1001.0)

smoothness_mean = st.number_input("Smoothness Mean", value=0.1184)

compactness_mean = st.number_input("Compactness Mean", value=0.2776)

concavity_mean = st.number_input("Concavity Mean", value=0.3001)

concave_points_mean = st.number_input("Concave Points Mean", value=0.1471)

symmetry_mean = st.number_input("Symmetry Mean", value=0.2419)

fractal_dimension_mean = st.number_input("Fractal Dimension Mean", value=0.07871)
```

```
36
37  fractal_dimension_mean = st.number_input("Fractal Dimension Mean", value=0.07871)
38
39  # Predict Button
40  if st.button("Predict"):
41      st.success("Input Received Successfully")
42
```

OUTPUT SCREENSHOT:



Q12. (Complete Streamlit Web App)

Complete the Streamlit application by:

- **Loading the saved model.**
- **Loading preprocessing objects.**
- **Accepting user input.**
- **Preprocessing the input.**
- **Displaying prediction using:**

Program Code:

```
#Questions 12
```

app_q12.py > ...

```python
# ============================================================
# Q12. Complete Streamlit Web App
# ============================================================

import streamlit as st
import pandas as pd
import joblib

# Load Saved Files
model = joblib.load("breast_cancer_model.pkl")
scaler = joblib.load("scaler.pkl")
columns = joblib.load("columns.pkl")

st.set_page_config(page_title="Breast Cancer Prediction")

st.title("🎗️ Breast Cancer Prediction System")

# Input Fields
radius_mean = st.number_input("Radius Mean", value=17.99)
texture_mean = st.number_input("Texture Mean", value=10.38)
perimeter_mean = st.number_input("Perimeter Mean", value=122.8)
area_mean = st.number_input("Area Mean", value=1001.0)
smoothness_mean = st.number_input("Smoothness Mean", value=0.1184)
compactness_mean = st.number_input("Compactness Mean", value=0.2776)
concavity_mean = st.number_input("Concavity Mean", value=0.3001)
concave_points_mean = st.number_input("Concave Points Mean", value=0.1471)
symmetry_mean = st.number_input("Symmetry Mean", value=0.2419)
fractal_dimension_mean = st.number_input("Fractal Dimension Mean", value=0.07871)

```

```python
# Predict Button
if st.button("Predict"):

    sample = pd.DataFrame([{
        "radius_mean": radius_mean,
        "texture_mean": texture_mean,
        "perimeter_mean": perimeter_mean,
        "area_mean": area_mean,
        "smoothness_mean": smoothness_mean,
        "compactness_mean": compactness_mean,
        "concavity_mean": concavity_mean,
        "concave points_mean": concave_points_mean,
        "symmetry_mean": symmetry_mean,
        "fractal_dimension_mean": fractal_dimension_mean
    }])

    # Add missing columns
    for col in columns:
        if col not in sample.columns:
            sample[col] = 0

    sample = sample[columns]

    sample = scaler.transform(sample)

    prediction = model.predict(sample)

    if prediction[0] == 1:
        st.error("🔴 Prediction: Malignant Tumor")
    else:
        st.success("🟢 Prediction: Benign Tumor")
```

OUTPUT SCREENSHOT: